

TriVoKo

Phase 7 - Flash sale

Time	Start date	Finish date	Status
1.5 weeks			Not started

Goal: Standout #1: a flash sale where thousands click at once, exactly the right number win, and stock never goes below zero - proven with a 5,000-user load test.

Real-life example: A big sale at a shop with 100 TVs and 5,000 people at the door. A guard gives out exactly 100 tokens, one per person. Token 101 does not exist.

Step checklist

★ = you type the key logic (Claude explains first, then reviews). "You" = only you can do it.

#	Step	Who	Done
1	Flyway: flash_sales, flash_sale_items, flash_sale_claims with UNIQUE(sale item, customer)	Claude	☐
2	Admin schedules a sale; sellers offer a variant with sale price + quantity; the quantity is taken out of normal stock	Claude	☐
3	★ You type the CLAIM: one atomic UPDATE ... SET sold = sold + 1 WHERE sold < quantity AND NOW() BETWEEN start AND end, the unique claim row, and the 10-minute hold at the sale price	★ Madhavan	☐
4	★ You type the GIVE-BACK: an unpaid claim returns its unit (sold = sold - 1)	★ Madhavan	☐
5	Sale end: unsold units go back to normal stock	Claude	☐
6	Rate limit on claim per user + IP (429 + Retry-After)	Claude	☐
7	Frontend: home banner countdown (server time), sale page, live "N left" counter, instant "Sold out"	Claude	☐
8	★ 1,000 threads / 100 units test - you write the final assertions	★ Madhavan	☐
9	k6 load test: 5,000 virtual users against the app in Docker; record requests per second, p95, winners and errors	Claude	☐
10	Decision: if p95 is above 500 ms, try Redis DECR and measure again; write the decision with both graphs	Together	☐

Output of this phase

- Flash sale feature with countdown and live counter
- Your claim + give-back code
- k6 report and graph in `load-tests\results`

Done when (check before moving on)

Check	Passed
k6 report: exactly 100 winners, 0 oversell, 0 errors	☐
p95 written in the README	☐

Explain it back (in your own words)

Question	Done
Why is "read stock, then write stock" wrong, and why does one UPDATE fix it?	☐
What stops one person from winning twice?	☐
What did k6 measure, and did we need Redis?	☐

Finish routine

Check	Done
All tests pass and CI is green	☐
Committed and pushed	☐
Done-when checks shown working in the browser	☐
Session summary + SESSION_LOG written	☐

Skills you practise: Concurrency, atomic SQL, unique constraints, rate limiting, load testing with k6

Where your work goes: `backend\...\flashsale`, `load-tests\`, flash-sale pages in `frontend`

My notes